📖 HashMap Mastery

Complete Visual Guide - From Diary to Code

Master Key-Value Pairs & String Problems

📚 Table of Contents

1. Introduction - The Diary Analogy
2. Java Syntax & Operations
3. First Unique Character (LC 387)
4. Ransom Note (LC 383)
5. Maximum Balloons (LC 1189)
6. Longest Palindrome (LC 409)
7. Summary & Patterns

1 Introduction - The Diary Analogy

📖 HashMap as a Diary

Imagine writing a diary where you record events on specific dates.

Later, you want to quickly remember: **"What happened on Dec 7?"**

📊 The Diary Example

```
Your Diary:
┌────────┬──────────────────┐
│  Date  │      Event       │  ← KEY : VALUE
├────────┼──────────────────┤
│  Dec 5 │  Bought a bike   │
│  Dec 6 │  Broke leg       │
│  Dec 7 │  Sold the bike   │
│  Dec 10│  Went to hospital│
└────────┴──────────────────┘

Question: "What happened on Dec 7?"
Answer: Look up key "Dec 7" → value is "Sold the bike"
```

🔑 Key-Value Pairs

Key (The Identifier):

- How you look up data (e.g., Date, Name, Character)
- Keys must be **UNIQUE**
- You decide what the key is

Value (The Data):

- The actual information stored
- Values can be **DUPLICATED**
- Can be any data type

✅ What HashMap CAN Do

- ✓ Tell you the **value** for a specific key
- ✓ Tell you if a **key exists** (True/False)
- ✓ Tell you the **size** (number of pairs)
- ✓ **Remember old data** for quick lookup
- ✓ **Track frequency** (character counts)

❌ What HashMap CANNOT Do

"Nonsense Questions":

✗ "What happened **before** Dec 10?" (range)

✗ "What was the **last** entry?" (order)

✗ "What's the **3rd** event?" (index)

Why? Standard HashMaps are **UNORDERED**

They don't maintain insertion order or sorting

🎯 Primary Use Case

HashMaps are used to REMEMBER OLD DATA
to answer: "What is the value for this Key?"

2 Java Syntax & Operations

📝 Declaration

```java
// HashMap requires Object types (Integer, not int)
HashMap<Integer, Integer> map = new HashMap<>();
HashMap<Character, Integer> freq = new HashMap<>();
HashMap<String, String> diary = new HashMap<>();
```

🔧 Common Operations

Operation	Code	Description
Insert/Update	`map.put(key, value)`	If key exists, updates value; else inserts
Get Value	`map.get(key)`	Returns value for key (or null if not found)
Get with Default	`map.getOrDefault(key, 0)`	Returns value or default if key missing
Check Exists	`map.containsKey(key)`	Returns true/false
Remove	`map.remove(key)`	Deletes the key-value pair
Size	`map.size()`	Returns number of pairs
Iterate Keys	`for (K key : map.keySet())`	Loop through all keys
Iterate Values	`for (V val : map.values())`	Loop through all values

🎯 Common Pattern: Frequency Counter

```java
// Building a frequency map (MOST COMMON!)
HashMap<Character, Integer> freq = new HashMap<>();

for (char c : string.toCharArray()) {
    // If c exists, get its count and add 1
    // If c doesn't exist, use 0 as default and add 1
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

3 Problem 1: First Unique Character

LeetCode 387: Find the first non-repeating character
Input: "leetcode"
Output: 0 (index of 'l')

🎯 Logic

1. **Build frequency map** - count all characters
2. **Check from start** - find first with count = 1
3. **Return index** or -1 if none found

📊 Visual Walkthrough

```
String:  L  E  E  T  C  O  D  E
Index:   0  1  2  3  4  5  6  7

Step 1: Build Frequency Map
┌─────┬───┬───┬───┬───┬───┬───┐
│ Key │ L │ E │ T │ C │ O │ D │
├─────┼───┼───┼───┼───┼───┼───┤
│ Val │ 1 │ 3 │ 1 │ 1 │ 1 │ 1 │
└─────┴───┴───┴───┴───┴───┴───┘

Step 2: Check from Index 0
i=0, char='L': map.get('L') = 1? YES! Return 0

(No need to check 'E' since we found unique character)
```

💻 Java Code (Fully Commented)

```java
/**
 * First Unique Character in a String
 * LeetCode 387
 *
 * Time: O(N) - two passes through string
 * Space: O(1) - at most 26 characters (English alphabet)
 */
public int firstUniqChar(String s) {
    // Step 1: Create HashMap for character frequency
    HashMap<Character, Integer> map = new HashMap<>();

    // Step 2: Build the frequency map
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        // If key exists, increment count; else insert 1
        map.put(c, map.getOrDefault(c, 0) + 1);
    }

    // Step 3: Find the first unique character
    // Must check from index 0 to maintain "first" order
    for (int i = 0; i < s.length(); i++) {
        if (map.get(s.charAt(i)) == 1) {
            return i;  // Return the index
        }
    }

    // Step 4: No unique character found
    return -1;
}
```

4 Problem 2: Ransom Note

LeetCode 383: Can construct ransom note from magazine?
Input: ransomNote = "aab", magazine = "baa"
Output: false (need 2 'a's but only have 2)

🎯 The Job Interview Analogy

Ransom = Requirement (NEED)

Magazine = Resume (HAVE)

For each requirement:

- If Have >= Need → ✓ Pass
- If Have < Need → ✗ Fail immediately
- Extra stuff in Magazine is ignored

📊 Visual Example

```
Ransom (NEED): "AAB"       Magazine (HAVE): "BAAXX"

Step 1: Build Maps
NEED Map:  { A: 2, B: 1 }
HAVE Map:  { B: 1, A: 2, X: 2 }

Step 2: Verification (Check NEED map only)
Check 'A':
  Need: 2
  Have: 2
  Is Have >= Need? YES ✓

Check 'B':
  Need: 1
  Have: 1
  Is Have >= Need? YES ✓

Result: TRUE (can construct)

Note: Extra 'X' in magazine is ignored!
```

💻 Java Code

```java
/**
 * Ransom Note
 * LeetCode 383
 *
 * Time: O(N + M)
 * Space: O(1) - at most 26 characters
 */
public boolean canConstruct(String ransomNote, String magazine) {
    HashMap<Character, Integer> have = new HashMap<>();
    HashMap<Character, Integer> need = new HashMap<>();

    // Step 1: Build NEED Map (Ransom)
    for (char c : ransomNote.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }

    // Step 2: Build HAVE Map (Magazine)
    for (char c : magazine.toCharArray()) {
        have.put(c, have.getOrDefault(c, 0) + 1);
    }

    // Step 3: Check requirements
    // Only iterate over NEED because we only care about requirements
    for (char key : need.keySet()) {
        int neededCount = need.get(key);
        // Use 0 as default if character not in magazine
        int haveCount = have.getOrDefault(key, 0);

        if (haveCount < neededCount) {
            return false;  // Not enough characters!
        }
    }

    return true;  // All requirements satisfied
}
```

5 Problem 3: Maximum Number of Balloons

LeetCode 1189: How many "balloon" can you form?
Input: "loonbalxballpoon"
Output: 2

🎯 The Slowest Worker Analogy

Factory Principle: Only as fast as slowest worker!

If you have:

- 100 wheels but only 8 engines → Make 8 cars
- 6 'L's but need 2 'L's per word → Make 3 words

Answer = Minimum ratio of (Have / Need)

📊 Visual Analysis

```
Input: "BBAALLOOOOONN"
Target Word: "BALLOON"

Step 1: Count Requirements for "balloon"
NEED: { b:1, a:1, l:2, o:2, n:1 }

Step 2: Count What We Have
HAVE: { B:2, A:2, L:2, O:5, N:2 }

Step 3: Calculate Ratio for Each Letter
┌────────┬──────┬──────┬─────────────┬──────────────┐
│ Letter │ Have │ Need │ Have / Need │ Max Possible │
├────────┼──────┼──────┼─────────────┼──────────────┤
│   B    │  2   │  1   │   2 / 1     │      2       │
│   A    │  2   │  1   │   2 / 1     │      2       │
│   L    │  2   │  2   │   2 / 2     │      1 ← 🔴  │
│   O    │  5   │  2   │   5 / 2     │      2       │
│   N    │  2   │  1   │   2 / 1     │      2       │
└────────┴──────┴──────┴─────────────┴──────────────┘

Bottleneck: 'L' (can only make 1 word)
Answer: Minimum = 1
```

💻 Java Code

```java
/**
 * Maximum Number of Balloons
 * LeetCode 1189
 *
 * Time: O(N)
 * Space: O(1)
 */
public int maxNumberOfBalloons(String text) {
    HashMap<Character, Integer> have = new HashMap<>();
    HashMap<Character, Integer> need = new HashMap<>();

    // Step 1: Build NEED Map for "balloon"
    String target = "balloon";
    for (char c : target.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }

    // Step 2: Build HAVE Map from input text
    for (char c : text.toCharArray()) {
        have.put(c, have.getOrDefault(c, 0) + 1);
    }

    int ans = Integer.MAX_VALUE;

    // Step 3: Calculate minimum ratio (bottleneck)
    for (char key : need.keySet()) {
        int neededCount = need.get(key);
        int haveCount = have.getOrDefault(key, 0);

        // How many times can THIS letter support the word?
        int times = haveCount / neededCount;

        // The answer is limited by the smallest ratio
        ans = Math.min(ans, times);
    }

    return ans;
}
```

6 Problem 4: Longest Palindrome

LeetCode 409: Length of longest palindrome from letters
Input: "abccccdd"
Output: 7 (e.g., "dccaccd")

🎯 Palindrome Rules

Even Counts: Use all (e.g., 4 'c's → "cc...cc")

Odd Counts: Use count-1 to make even (e.g., 5 'a's → use 4)

The Center: Can place ONE leftover in middle (+1)

Key: Palindromes need matching pairs!

📊 Visual Walkthrough

```
Input: "AAABBBCCCCD"
Frequency Map: { A:3, B:3, C:4, D:1 }

Processing Each Character:

'C' (count=4): EVEN
  Use all 4 → Total = 4

'A' (count=3): ODD
  Use 3-1 = 2 → Total = 4 + 2 = 6
  (1 'A' left over)

'B' (count=3): ODD
  Use 3-1 = 2 → Total = 6 + 2 = 8
  (1 'B' left over)

'D' (count=1): ODD
  Use 1-1 = 0 → Total = 8
  (1 'D' left over)

Final Step:
Had odd counts? YES (A, B, D leftovers)
Can put ONE in center → Total = 8 + 1 = 9

Example palindrome: "CCAABBDACC" (or similar)
```

💻 Java Code

```java
/**
 * Longest Palindrome
 * LeetCode 409
 *
 * Time: O(N)
 * Space: O(1)
 */
public int longestPalindrome(String s) {
    HashMap<Character, Integer> map = new HashMap<>();

    // Step 1: Build Frequency Map
    for (char c : s.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }

    int result = 0;
    boolean hasOdd = false;

    // Step 2: Process each character frequency
    for (int val : map.values()) {
        if (val % 2 == 0) {
            // Even: Use all characters
            result += val;
        } else {
            // Odd: Use (val - 1) to make it even
            result += (val - 1);
            // Mark that we have at least one odd count
            hasOdd = true;
        }
    }

    // Step 3: If we had any odd counts, add 1 for center
    if (hasOdd) {
        return result + 1;
    } else {
        return result;
    }
}
```

7 Summary & Patterns

📊 Problem Patterns Summary

Problem	Pattern	Key Concept
First Unique Character	Frequency + Linear Scan	Find first with count=1
Ransom Note	Two Maps: Need vs Have	Check Have >= Need
Maximum Balloons	Ratio Calculation	Min(Have/Need) = bottleneck
Longest Palindrome	Even/Odd Processing	Pairs + optional center

🎯 Common Template

```java
// Step 1: Build frequency map
HashMap<Character, Integer> map = new HashMap<>();
for (char c : string.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}

// Step 2: Process based on problem type
for (char key : map.keySet()) {
    int count = map.get(key);
    // Apply logic here
}
```

💡 Key Insights

1. **Frequency Counting:** Most common use in strings
2. **Need vs Have:** Compare requirements to availability
3. **Bottleneck:** Minimum ratio determines answer
4. **Even/Odd:** Important for palindrome/pairing problems
5. **O(1) Space:** Fixed alphabet size (26 chars)

🎉 You've Mastered HashMap!

You now understand:

- Key-Value pair concept
- Frequency counting pattern
- Need vs Have comparison
- Ratio calculations
- All 4 classic string problems!

Practice these to solidify your skills!